

Lab 3: Component Modelling & Architectural Pattern Selection

Student Name: Sahajpreet Singh | **SRN:** PES1UG24CS397 | **Section:** G

GitHub Repository Link: <https://github.com/sahajsidhu78-Sahaj/SE-Labs-PES1UG24CS397> (folder: Lab3)

1. Scenario Overview & Architectural Selection

Problem Statement #29: Municipal Infrastructure Damage Reporting App

Domain: Smart Cities, Transport & Logistics | **Primary Actors:** Citizen Reporter, Municipal Engineer

Scenario description: Citizens photograph damaged public infrastructure (potholes, broken streetlights, water leaks) using a mobile app. The system reads the GPS coordinates embedded in the photo's EXIF metadata to place the incident on the municipal GIS map, generates a ticket, and automatically routes it to the correct engineering department. Municipal engineers update the ticket status as they work, and the citizen is notified of every change. Because mobile coverage in the field is unreliable, reports can be drafted offline and are synchronised automatically when connectivity returns.

Architecture Selection: "We chose **Layered Architecture** (3-tier: Presentation Layer, Business Layer and Data Layer) for the **Municipal Infrastructure Damage Reporting App** System."

2. Requirements Review & Main Challenges

Type	Requirement	Drives these components
Functional	Extract the GPS coordinates from the EXIF metadata of an uploaded damage photo to pinpoint the incident on the municipal GIS map.	Location Service, Report Manager
Functional	Let the Citizen Reporter select a damage category (pothole, streetlight, water leak) when submitting a report.	Reporting UI, Report Manager
Functional	Automatically dispatch a generated ticket to the relevant engineering department based on the damage category.	Report Manager, Database
Functional	Let a Municipal Engineer update the ticket status (Open, In Progress, Resolved).	Reporting UI (engineer dashboard), Report Manager
Functional	Notify the Citizen Reporter when the status of their ticket changes.	Notification Service, Report Manager
Non-functional	Support offline report drafting and auto-sync when mobile connectivity resumes (performance and security).	Reporting UI, Report Manager (IReportService)
Non-functional	Geotagged photo uploads should complete reliably within 10 seconds on a 4G connection (usability / reliability).	Report Manager, IReportService

Main challenges identified

- **Performance:** photo uploads must meet the 10-second target on mobile networks, and report volume can spike after storms or floods.
- **Usability / reliability:** citizens must be able to draft a report with no signal and have it synchronise later without losing the photo or the location.
- **Security / privacy:** photos and GPS coordinates are personal data, and the two roles (citizen and engineer) must only be able to do what their role allows.
- **Correct routing:** every category must reach the right department, and a photo without GPS data must fall back to a manually pinned location.
- **Maintainability:** the GIS provider and the push/SMS gateway are external and may change, so they must be isolated behind interfaces.

3. Architectural Style Analysis

Style	Key pros	Key cons	Performance & security impact here	Suitability
Layered (Selected)	Clear separation of UI, business rules and data; each layer can be built and tested separately; simple to understand and deploy; one central place to enforce access control.	Requests must pass through the layers (small overhead); adjacent layers are coupled; scaling is per tier, not per feature.	Short request path (UI → Report Manager → Database). The database is never exposed to the app. The Report Manager tier can be replicated behind a load balancer.	Best fit. Matches the one-directional report workflow and a small project team.
Microservices	Independent deployment and scaling (e.g. photo upload, dispatch, notification); fault isolation; technology diversity.	Operational complexity (service discovery, monitoring, one pipeline per service); network latency between services; data consistency across report and ticket state.	Extra network hops on the submit path put the 10 s upload target at more risk; many separate endpoints enlarge the attack surface.	Poor fit. Too heavy for five components and a small municipal IT team.
Client-Server	Centralised control, simple deployment, easy data consistency; naturally describes mobile apps talking to one central server.	Server is a single point of failure and a scaling bottleneck; limited fault tolerance; says nothing about how the server itself is organised.	One server handling every upload and notification may bottleneck after a storm; central control is good for security, but offers no internal separation.	Partial fit. Valid as the deployment view (mobile client ↔ server), but the server still needs a layered structure.

Decision: Layered Architecture is selected. The system is small (five components), its workflow is a clear top-down chain (capture → validate and locate → store and dispatch → notify), and the main risks (privacy, reliability of uploads) are handled better by clear layer boundaries than by distributing the system.

Component identification (Step 3)

With the layered style chosen, the components were identified layer by layer from the requirements above. The Report Manager plays the role of the handout's Order Manager (central orchestrator), and the Location Service takes the place of the Payment Service as the specialised service it calls.

Layer	Component identified	Why it exists (requirement it satisfies)
Presentation	Reporting UI	Citizens need a mobile front end for category selection, photo capture and offline drafts; engineers need a dashboard to view and update tickets.
Business	Report Manager	Validates reports, creates and dispatches tickets by category, applies status changes and role rules; the one component every other business component is coordinated by.
Business	Location Service	Isolates EXIF GPS extraction and GIS mapping, so the GIS provider can change without touching the report workflow.
Business	Notification Service	Isolates push/SMS delivery so citizens are told about every status change and the gateway can be replaced independently.
Data	Database	Stores reports, photo metadata, coordinates, tickets, status history and routing rules; reachable only through IDataAccess.

Architectural Justification

System: Municipal Infrastructure Damage Reporting App

Architecture Selection

“We chose **Layered Architecture** for the **Municipal Infrastructure Damage Reporting App** System.”

Architectural Choice

The system is organised in three layers: a **Presentation Layer** (Reporting UI, serving both the Citizen Reporter app and the Municipal Engineer dashboard), a **Business Layer** (Report Manager, Location Service, Notification Service) and a **Data Layer** (Database). The Presentation Layer depends only on the Business Layer, and only the Business Layer depends on the Data Layer. Inside the Business Layer, the Report Manager is the orchestrator and is the only component that calls the Location Service, the Notification Service and the Database. The four interfaces are `IReportService`, `ILocationService`, `INotification` and `IDataAccess`.

Two Reasons

1. The layers match the way a damage report actually flows. A report moves in one direction: the citizen captures a photo and category (Reporting UI); the Report Manager validates it, asks the Location Service for the EXIF GPS position, creates the ticket and dispatches it to the right department; the Database stores everything; and the Notification Service informs the citizen when the status changes. Each step is owned by exactly one component, so the work could be split across the team without stepping on each other. The two roles share one Report Manager and differ only in the UI, and the GIS provider or the SMS/push gateway can be replaced by changing one component without touching the rest.

2. It is right-sized for a municipal project. The scope is five components and four interfaces. A layered design keeps development, testing and deployment straightforward for a small city IT team, whereas microservices would add service discovery, distributed data consistency and extra network hops that this system does not need. Client-server alone is too coarse: it describes the mobile-app-to-server split, but not how the server is structured internally.

Security Advantage

Every request from the Citizen or Engineer UI has to pass through the Report Manager’s **IReportService** interface, which gives one place to enforce authentication and role-based access: a citizen can only create reports and view their own, and only a Municipal Engineer can change a ticket’s status. The Database is reachable only through **IDataAccess** from the business layer, so stored photos, GPS coordinates and contact details cannot be read or changed by bypassing business rules, and the mobile app never holds database credentials. Traffic between the mobile app and the Report Manager is sent over HTTPS/TLS.

Performance Benefit

The Report Manager is stateless, so several instances can run behind a load balancer when the report volume spikes (for example after a storm). Slower work, namely GPS extraction and GIS lookup and the delivery of notifications, sits behind separate interfaces and can run asynchronously, so the citizen receives a ticket ID quickly and the 10-second upload target is not held up by notification delivery. Frequently read data such as department routing rules can be cached in the business layer to reduce database round trips.

Trade-off Accepted

Passing through layers adds a small amount of overhead, and adjacent layers are coupled through their interfaces. This is acceptable because the request path is short (UI → Report Manager → Database) and the interfaces are kept small and stable.

UML Component Diagram — Municipal Infrastructure Damage Reporting App

Problem Statement #29 | Architectural pattern: 3-tier Layered Architecture | 5 components, 4 interfaces

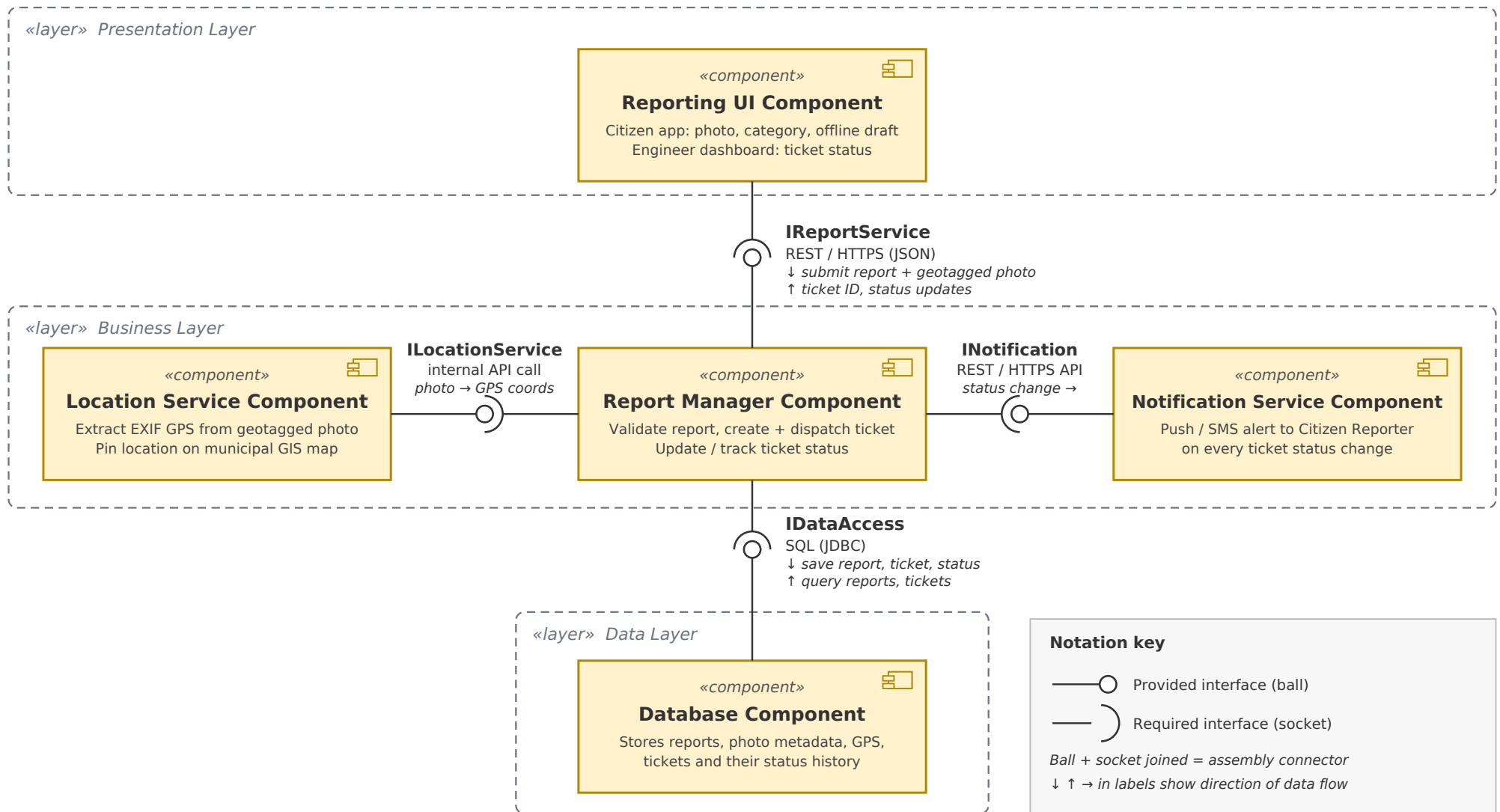


Figure 1: Component diagram. A ball marks a provided interface (what the component offers); a socket marks a required interface (what it needs). A ball joined to a socket is an assembly connector. All four interfaces centre on the Report Manager, which orchestrates the other components.

4. Exactly 5 Components — Breakdown

Component	Layer	Detailed responsibility	Provides / requires
Reporting UI Component	Presentation	Citizen app: select damage category, capture or upload a geotagged photo, add a description, save offline drafts locally and auto-sync them when connectivity returns, and track report status. Engineer dashboard: list tickets dispatched to the engineer's department and update status (Open, In Progress, Resolved). Has no direct database access.	Requires IReportService
Report Manager Component	Business	Central orchestrator. Authenticates requests and applies role-based rules (citizen vs engineer); validates each submission (photo, category, location); obtains coordinates from the Location Service and, if the photo has no GPS data, asks the UI for a manual map pin; creates the ticket with a unique ID; routes it to the department for the chosen category; applies status changes; triggers notifications; and persists everything through the Database.	Provides IReportService. Requires ILocationService, INotification, IDataAccess
Location Service Component	Business	Reads the EXIF GPS metadata from the uploaded photo, reports clearly when none is present, and maps the coordinates onto the municipal GIS map (road, ward) so the ticket carries an exact location.	Provides ILocationService (uses the external municipal GIS internally)
Notification Service Component	Business	Builds and sends a push/SMS message to the Citizen Reporter whenever their ticket status changes. Wraps the external gateway so the provider can be changed without affecting other components, and retries failed sends.	Provides INotification (uses the external push/SMS gateway internally)
Database Component	Data	Persistent storage for reports, photo references and metadata, GPS coordinates, tickets and their status history, department routing rules, and user accounts with roles.	Provides IDataAccess

5. Exactly 4 Interfaces — Specification

#	Interface	Provided (ball)	Required (socket)	Technology	Interaction and data flow
1	IReportService	Report Manager	Reporting UI	REST / HTTPS (JSON)	UI sends the report (photo, category, description), saved-draft syncs and status updates; Report Manager returns the ticket ID, current status and validation errors. Operations: submitReport, syncDrafts, getReportStatus, updateTicketStatus.
2	ILocationService	Location Service	Report Manager	Internal API call	Report Manager passes the photo; Location Service returns the GPS coordinates and GIS location, or a "no GPS data" result. Operation: extractLocation.
3	INotification	Notification Service	Report Manager	REST / HTTPS API	Report Manager sends the citizen ID, ticket ID and new status on every status change; Notification Service returns a delivery result. Operation: sendStatusNotification.
4	IDataAccess	Database	Report Manager	SQL (JDBC)	Report Manager saves and queries reports, tickets, status history and routing rules, and receives the stored records back. Operations: save, query, update.

Component interaction for "Submit Damage Report"

1. The citizen selects a category and captures a photo in the Reporting UI; if there is no signal, the report is saved as a local draft and sent later.
2. The UI calls **submitReport** on IReportService over HTTPS.
3. The Report Manager authenticates the citizen and validates the submission.
4. The Report Manager calls **extractLocation** on ILocationService, which reads the EXIF GPS data. If there is no GPS data, the UI is asked for a manual map pin.
5. The Report Manager creates the ticket, picks the department for the category and stores the report and ticket through IDataAccess.
6. The ticket ID is returned to the UI through IReportService, and the ticket appears in the engineer's queue.
7. When the engineer updates the status, the UI calls IReportService; the Report Manager saves it through IDataAccess and calls INotification so the citizen is alerted.

6. Final Checklist

Status	Checklist item	Verification notes
✓	Exactly 5 components are present.	Reporting UI, Report Manager, Location Service, Notification Service, Database.

Statu s	Checklist item	Verification notes
✓	Correct UML component notation is used.	«component» rectangles with the component icon and the name at the centre, grouped into layer packages.
✓	Exactly 4 required interfaces are shown.	IReportService, ILocationService, INotification, IDataAccess.
✓	Interface directions and labels are clear.	Each interface is labelled with its technology and the direction of data flow (Figure 1 and Section 5).
✓	Provided / required interface notation is used appropriately.	Ball on the providing component, socket on the consuming component, joined as assembly connectors.
✓	Architecture choice is clearly justified.	Section 3 and the one-page justification.
✓	Written justification is a maximum of 1 page.	It sits on its own single page.
✓	Diagram exported as PNG / PDF.	component_diagram.png, and Figure 1 in this PDF.
✓	Justification saved as PDF.	Included in this PDF (Architectural Justification page).
□	Both deliverables submitted through the GitHub repository.	To do: upload this PDF and component_diagram.png to the Lab3 folder of SE-Labs-PES1UG24CS397 and confirm the repository is public.